Що таке патерн Flyweight?

Flyweight – це структурний патерн проектування, який дозволяє економити оперативну пам'ять, коли програмі потрібно створити дуже багато схожих об'єктів. Замість того щоб кожен раз створювати новий об'єкт з нуля, патерн виділяє спільну частину даних (внутрішній стан) і зберігає її в одному екземплярі, який потім перевикористовується. Унікальні дані (зовнішній стан) передаються ззовні при виклику методів.

Простіше кажучи: якщо у нас є 1000 автомобілів BMW M5 червоного кольору, нам не потрібно зберігати марку, модель і колір 1000 разів – достатньо зберегти це один раз і посилатися на цей об'єкт.

Як це реалізовано у моєму проєкті

Проєкт складається з трьох основних файлів:

1. flyweight.py – клас Flyweight

Цей клас зберігає **спільний (внутрішній) стан** об'єкта. У конструкторі __init__ він приймає список і перетворює його в незмінний кортеж (tuple), щоб ніхто не міг випадково змінити спільні дані. Метод operation() приймає унікальний стан ззовні і виводить обидва стани разом.

```
class Flyweight:

    def __init__(self, shared_state: list) -> None:

        self._shared_state = tuple(shared_state)



    def operation(self, unique_state: str) -> None:

        print(f"Flyweight:     shared     ({self._shared_state})     +     unique
({unique_state}) state.")
```

2. flyweight_factory.py – клас FlyweightFactory

flyweight_factory – це головний механізм патерну. Вона зберігає словник (dict) вже створених легковаговиків. Коли клієнт просить об'єкт через метод get_flyweight(), фабрика спочатку перевіряє, чи є вже такий об'єкт у кеші.

Якщо є – повертає існуючий, якщо ні – створює новий і додає його в словник. Ключ кешу формується через метод _get_key(), який сортує елементи стану і з'єднує їх через підкреслення.

```
def get_flyweight(self, shared_state: list) -> Flyweight:

    key = self._get_key(shared_state)

    if key not in self._flyweights:

        self._flyweights[key] = Flyweight(shared_state)  # створюємо новий

    else:

        pass  # повертаємо існуючий з кешу

    return self._flyweights[key]
```

3. main.py – клієнтський код

У main.py ми створюємо фабрику і заповнюємо її початковими типами автомобілів (Chevrolet Camaro, Mercedes C300, BMW M5, BMW X6). Далі через функцію add_car_to_database() додаємо нові авто в базу. Ця функція розділяє дані на:

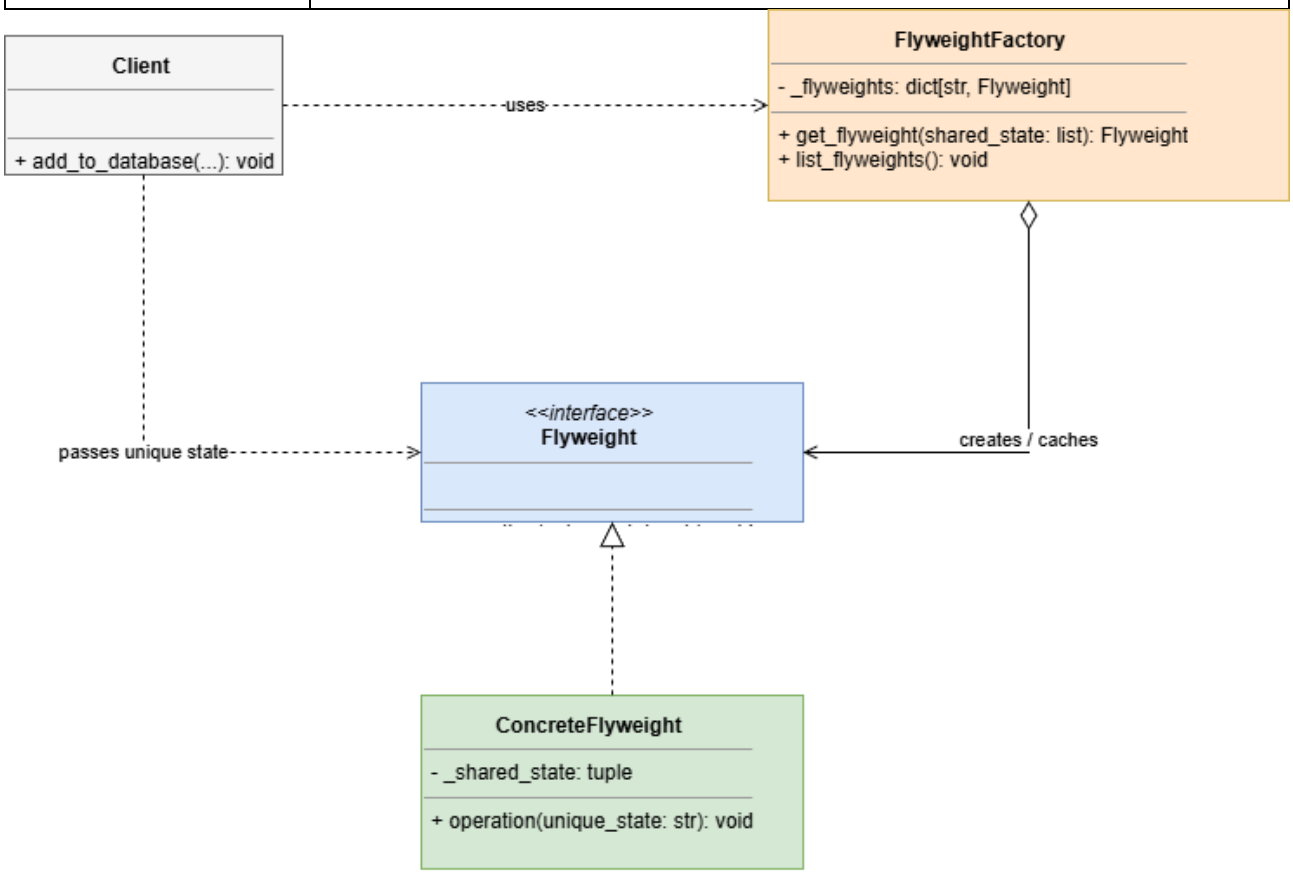
- Спільний стан (марка, модель, колір) – передається у фабрику.
- Унікальний стан (номерний знак, власник) – передається в метод operation().

Коли ми додаємо ще одне BMW M5 червоного кольору, фабрика не створює новий об'єкт, а повертає вже існуючий – це і є суть патерну Легковаговик.

Підсумок

Компонент	**Роль у патерні**
Flyweight	Зберігає спільний стан, надає метод для роботи із зовнішнім станом
FlyweightFactory	Керує створенням та кешуванням легковаговиків

Компонент	Роль у патерні
main.py (Клієнт)	Використовує фабрику, передає зовнішній стан при виклику операцій



Таким чином, завдяки патерну Flyweight ми уникаємо дублювання даних і значно зменшуємо використання пам'яті при роботі з великою кількістю подібних об'єктів.